

The Blueprint, Part 2: रचनेचे निर्णय

August 30, 2026

THE BLUEPRINT

Book 5, Part 2: रचनेचे निर्णय

मागच्या भागाचा शेवट: आरतीचे दोन मोठे निर्णय टेबलावर: app चे तुकडे करावेत का, आणि कडक database की सैल. हा भाग अशा **मोठ्या रचना-निर्णयांचा** आहे: जिथे “भारी tech” चा मोह सर्वात जास्त, आणि सूत्र सर्वात जास्त उपयोगी.

सूत्र (पुन्हा)

मोठेपणा फुकट नसतो.
प्रत्येक रचना-निर्णय म्हणजे काहीतरी विकून काहीतरी विकत घेणं.
Trade-off न दिसणं हीच खरी चूक. (आणि: fashion नको, सौदा बघा.)

कपाटावर एक नजर

Book 1	The Machine	Book 2	The Thinking Machine
Book 3	The Land Game	Book 4	The Harness
Book 5	The Blueprint	<- हे, Part 2: रचना-निर्णय	

तीन भागांचा नकाशा: तुम्ही इथे आहात

PART 1	प्रवास: एक user ते कोटी	(झाला)
PART 2	रचनेचे निर्णय	<- तुम्ही इथे
	data-रचना, SQL/NoSQL, DB आतून, API, monolith/microservices, search, real-time, migrations	
PART 3	Elite शिस्त + खऱ्या इमारती	(पुढे)

Part 1 ची उजळणी: 4 प्रश्न (बोलून; उत्तरं खाली)

1. App हळू: पहिलं निदान काय, “मोठा server” का नाही?

| रचना: गहाळ index/cache/replica. 90% हळूपणा रचनेत, ताकदीत नाही. (1.3-1.5)

2. Cache/CDN चा खरा अवघड प्रश्न?

| "जुनं कधी फेकायचं" (invalidation); वेग विकत, ताजेपणा विकून. (1.4, 1.9)

3. पैशाला CAP मध्ये C की A?

| C: फूट पडली तर थांबा, पण चुकीचा आकडा नाही. (1.10)

4. System-आरोग्य सरासरीने का नाही?

| टोक (p99) खरं; सरासरी अल्पसंख्य-दुःखी लपवते. (1.12)

या भागात काय

Data कशी रचायची, SQL-vs-NoSQL चा गाजलेला वाद, database आतून कशी शोधते-लिहिते, API म्हणजे करार, monolith-vs- microservices (आणि Amazon Prime Video ची उलटी गोष्ट), search कशी चालते, real-time (WhatsApp), आणि चालत्या system चं चाक बदलणं (migrations). प्रत्येक निर्णयाला तेच सूत्र: सौदा बघा, fashion नको.

विभाग 2: मोठे रचना-निर्णय

Part 1 चे सौदे छोट्या-छोट्या भिंतींचे होते. या भागातले निर्णय मोठे आहेत: एकदा घेतले की app चं भवितव्य वर्षानुवर्ष ठरवतात, आणि बदलायला महाग. **हे तुमच्या धंद्यात कुठे येईल:** इथेच “भारी नवं tech वापरू” चा मोह सर्वात जास्त, आणि तिथेच सर्वात जास्त पैसा-वेळ वाया जातो. हा भाग तुम्हाला fashion आणि सौदा वेगळे करायला शिकवेल: top engineer ची सर्वात मोठी ओळख.

Chapter 2.1 [SPINE]: Data कशी रचायची: Data Modeling

Part 2 चा पाया: माहिती **कशी मांडायची**. हा निर्णय सर्वात आधीचा आणि सर्वात टिकाऊ: कारण एकदा लाखो नोंदी एका रचनेत साठल्या की ती बदलणं म्हणजे चालत्या गाडीचं चाक बदलणं (2.11 चा विषय). म्हणून आधी नीट.

Data model म्हणजे काय. दुकानाच्या वहांची उपमा: कोणती माहिती कोणत्या वहीत, कोणते रकाने, आणि वहा एकमेकांशी कशा जोडलेल्या. आरतीच्या app ची माहिती: गिन्हाईक, दुकान, वस्तू, ऑर्डर. प्रश्न: या चार गोष्टी कशा मांडायच्या की (अ) माहिती **एकाच जागी** राहील (दुरुस्ती एकाच ठिकाणी) आणि (ब) हवं ते **जलद** मिळेल. या दोन इच्छांमध्ये सतत ताण आहे, आणि तोच data modeling चा गाभा.

पहिलं तत्त्व: एक सत्य, एक जागा (normalization). रमेशचा पत्ता प्रत्येक ऑर्डरमध्ये पुन्हा-पुन्हा लिहायचा का? नाही: रमेशची एक “गिन्हाईक-नोंद”, त्यात पत्ता एकदाच; ऑर्डर फक्त रमेशच्या **क्रमांकाकडे बोट** करते. फायदा: रमेशने पत्ता बदलला तर **एकाच** जागी बदल, सगळ्या ऑर्डरना लागू. जर पत्ता प्रत्येक ऑर्डरमध्ये कॉपी केला असता, तर 500 ऑर्डरमध्ये 500 जागी बदलावा लागला असता (आणि एखादी राहिली की चुकीचा पत्ता: data मधली विसंगती). हे “पुनरावृत्ती टाळणं, बोट-दाखवणं” म्हणजे normalization: **एक सत्य, एकाच जागी**.

पण इथेही सूत्र: normalization = शुद्धता विकत, वेग विकून. रमेशची ऑर्डर + त्याचा पत्ता + दुकानाचं नाव दाखवायला आता **तीन वहा जुळवाव्या** लागतात (ऑर्डर -> गिन्हाईक -> दुकान: याला “join” म्हणतात): शुद्ध, पण हळू. म्हणून कधीकधी जाणीवपूर्वक **उलट** केलं जातं: वारंवार-लागणारी माहिती मुद्दाम कॉपी करून एकत्र ठेवणं (denormalization): वेग मिळतो, पण आता “एक सत्य” सुटतं (बदल अनेक जागी). निर्णय कशावर? **वाचन जास्त की लेखन जास्त, आणि विसंगती किती परवडते:** पैसे-नोंदींना शुद्धता (normalize); वारंवार-दाखवायच्या याद्यांना वेग (थोडं denormalize). पुन्हा तोच index-चा (1.3) आत्मा: वाचन-वि.-लेखन चा सौदा, आता रचनेच्या पातळीवर.

दुसरं तत्त्व: नाती ओळखा. एका गिन्हाईकाच्या **अनेक** ऑर्डर (one-to-many); एका ऑर्डरमध्ये **अनेक** वस्तू आणि एक वस्तू **अनेक** ऑर्डरमध्ये (many-to-many). ही नाती नीट ओळखून मांडणं म्हणजे अर्ध data modeling. चुकीचं नातं मांडलं (उदा एका ऑर्डरला एकच वस्तू, नंतर गरज पडली अनेक) की पुढे पूर्ण रचना मोडून पुन्हा बांधावी लागते.

महाग चूक

”आधी code लिहू, data-रचना नंतर बघू.” उलटं शहाणपण: **data- रचना आधी, नीट**. कारण code बदलणं सोपं (आज लिहिलं, उद्या बदललं), पण **लाखो नोंदी साठल्यावर रचना बदलणं** म्हणजे महिन्यांचं, धोक्याचं काम (2.11). Twitter, Facebook: सगळ्यांनी सुरुवातीच्या घाईच्या data-रचनेची किंमत वर्षानुवर्ष भरली. Data- model वर एक दिवस जास्त घालवणं = पुढे महिने वाचवणं. “नाती” आधी कागदावर काढा, मग code.

नकाशावर

Part 2 चा नकाशा (वाढत जाईल):

2.1 data modeling: एक सत्य एका जागी (normalize) vs वेगासाठी कोपी (denormalize); नाती आधी ओळखा; रचना आधी, नीट

सूत्रावर: data-रचना हा सर्वात टिकाऊ सौदा: शुद्धता की वेग, आणि तो बदलायला सर्वात महाग. म्हणून इथे घाई सर्वात महाग, विचार सर्वात स्वस्त.

स्वतः बघा (5 मिनिटं)

कागदावर तुमच्या ओळखीच्या धंद्याची (दुकान, वर्ग, काहीही) माहिती चार पेट्यांत मांडा आणि बाणांनी नाती जोडा: कोण-कोणाशी one-to-many, कोण many-to-many. एखादी माहिती दोन जागी लिहावी लागतेय का बघा: तिथे normalization चा निर्णय आहे. हा साधा व्यायाम म्हणजे data modeling चा गाभा.

विचार करा

आरतीने विचारलं: “माझी ऑर्डर-नोंदी कडक database (PostgreSQL) मध्ये, ठीक. पण ‘रमेशने आज कोणकोणती दुकानं बघितली’, ‘कुठल्या वस्तूवर किती वेळा click’, अशी प्रचंड, सैल, दर सेकंदाला वाढणारी माहिती त्याच कडक, ACID database मध्ये कशाला कोंबू? तिथे शुद्धतेपेक्षा वेग-आणि-आकार हवा.”

अगदी नेमकं बोट. सगळी माहिती एकाच प्रकारच्या database मध्ये कोंबणं ही चूक; माहितीच्या स्वभावानुसार database ही वेगळी हवी. आणि इथूनच software मधला सर्वात गाजलेला वाद जन्मतो: SQL vs NoSQL. पुढचा chapter.

Chapter 2.2 [SPINE]: SQL vs NoSQL: गाजलेला वाद [दोन्ही बाजू]

Software मधला सर्वात गाजलेला, आणि सर्वात गैरसमजलेला वाद. आरतीच्या प्रश्नातून तो नीट उलगडू: आणि नेहमीप्रमाणे, उत्तर “कुठलं भारी” नसून “कधी कुठलं” असेल.

दोन घराणी. (1) **SQL / relational** (PostgreSQL, MySQL): माहिती **ठरलेल्या रकान्यांच्या टेबलांत**, नाती जोडलेली (2.1), आणि ACID ची कडक शिस्त (1.2). सरकारी फॉर्मची उपमा: प्रत्येक रकाना ठरलेला, शिस्त कडक, चूक होत नाही, पण साचा बदलायला अर्ज-परवानग्या. (2) **NoSQL** (MongoDB, Cassandra, Redis वगैरे: एक नाव नाही, अनेक प्रकार): माहिती **सैल, लवचिक** रूपात (प्रत्येक नोंद वेगळ्या आकाराची चालते), प्रचंड आकाराला- वेगाला सोयीची, पण कडक शिस्त सैल. कोन्या वहीची उपमा: काहीही, कसंही लिहा, वेगवान, पण शिस्त तुमच्यावर.

खरा फरक एका ओळीत: SQL शिस्त-आणि-नाती देतं (शुद्धता, गुंतागुंतीचे प्रश्न सोपे), वेग-लवचिकता विकून; NoSQL आकार- आणि-वेग देतं, शिस्त-नाती विकून. हा तोच जुना ACID-सौदा (1.2), आता database-निवडीत. आरतीचं उत्तर आता स्पष्ट: **ऑर्डर-पैसे-शिल्लक (नाती, शुद्धता हवी): SQL; click-log, बघितलेली-दुकानं (प्रचंड, सैल, वेग हवा): NoSQL.** एकाच app मध्ये दोन्ही: माहितीच्या स्वभावानुसार. याला “polyglot persistence” म्हणतात: अनेक database-भाषा एकत्र.

आणि आता 2026 चा कडू निकाल (हाच contested-गाभा):

NoSQL-उत्साहाची लाट (2010 च्या दशकात): “SQL जुनं, NoSQL सगळीकडे वापरा, तेच भविष्य!”
 प्रत्यक्ष अनुभव (आजवरचा): बहुतेकांना NoSQL ची गरज नव्हती; त्यांनी नाती (joins), शिस्त (ACID) गमावली आणि नंतर ती हाताने, code मध्ये, वाईट रीतीने पुन्हा बांधावी लागली.
 2026 ची स्थिती: PostgreSQL (SQL) हे developers मध्ये सलग सर्वात लोकप्रिय database (Stack Overflow पाहणी). आणि गंमत: आधुनिक SQL आता NoSQL ची लवचिकताही (JSON रकाने) देतं. “SQL मेलं” ही भविष्यवाणी उलटली.
 निकाल: NoSQL चुकीचं नाही; तो ****विशिष्ट**** गरजांसाठी (प्रचंड आकार, सैल-रचना, अति-वेग) उत्तम. पण ****डिफॉल्ट**** SQL असावं; NoSQL तेव्हा जेव्हा SQL खरेच कमी पडतं, fashion म्हणून नाही.

निवडीची कसोटी (छापून ठेवा): (1) माहितीला **नाती + शुद्धता** लागतात का (पैसे, ऑर्डर, नोंदी)? -> SQL. (2) माहिती **प्रचंड, सैल-आकाराची, अति-वेगवान** आहे आणि नाती कमी (logs, sensor- data, cache)? -> NoSQL. (3) **शंका असेल तर SQL:** कारण नाती नंतर सोडता येतात, पण नसताना पुन्हा

बांधणं महाग.

महाग चूक

”मोठं, आधुनिक app म्हणजे NoSQL; SQL जुन्या लोकांचं.” हा fashion-निर्णय हजारो startups ला महागात पडला: त्यांनी सुरुवातीलाच NoSQL निवडलं (कारण “cool”), मग कळलं की त्यांना ऑर्डर-गिन्हाईक-payment मधली नाती हवीच होती (जी SQL फुकट देतं), आणि ती त्यांनी code मध्ये हाताने, buggy रीतीने बांधली. **Database ची निवड “काय cool आहे” वर नाही, “माझ्या माहितीला काय लागतं” वर.** Fashion आणि सौदा वेगळे करणं: हीच या chapter ची, आणि पूर्ण Part 2 ची, खरी परीक्षा.

नकाशावर

- 2.1 data modeling: एक सत्य एका जागी; नाती आधी
- 2.2 SQL vs NoSQL: SQL=शिस्त+नाती, NoSQL=आकार+वेग;
डिफॉल्ट SQL, NoSQL गरजेला (fashion नको); दोन्ही एकत्र
(माहितीगणिक)

सूत्रावर: हा वाद म्हणजे सूत्राची शुद्ध परीक्षा: “भारी tech” चा मोह विरुद्ध “माझ्या माहितीचा सौदा.” जो सौदा बघतो तो बरोबर निवडतो; जो fashion बघतो तो दोनदा बांधतो.

स्वतः बघा (5 मिनिटं)

तुमच्या ओळखीच्या माहितीचे दोन प्रकार शोधा: एक जिथे **अचूकता आणि नाती** महत्त्वाची (बँक-खातं, विद्यार्थ्यांचे गुण), एक जिथे **प्रचंड आकार आणि वेग** पण अचूकता दुय्यम (कोणी काय बघितलं, किती वेळ). पहिल्याला SQL, दुसऱ्याला NoSQL: का, ते स्वतःला समजावा. ही जोडी ओळखता येणं = database-नजर.

विचार करा

आरतीला कुतूहल: “SQL ‘नाती जुळवून’ शोधतं, NoSQL ‘सैल-वेगवान’, हे कळलं. पण **आतून** काय वेगळं करतात? म्हणजे index (1.3) तर दोन्हीत असेल. एकच ‘शोध’ दोघांत इतका वेगळा का पडतो? database प्रत्यक्ष disk वर लिहिते-शोधते कशी?”

हाच पुढचा, थोडा खोलातला पण सुंदर chapter: database चं हृदय: B-tree आणि LSM-tree, दोन वेगळ्या लिहिण्या-शोधण्याच्या रीती, आणि त्यातून वाचन-वि.-लेखन चा तोच सौदा, आता सर्वात खोल पातळीवर.

Chapter 2.3 [DEPTH]: Database चं हृदय: B-tree आणि LSM-tree

[DEPTH]: हा chapter database च्या तळघरातला आहे. घाई असेल तर शेवटचा सौदा-ठोकळा वाचा. पण थांबलात तर “वाचन-वि.-लेखन” हा सौदा त्याच्या सर्वात खोल, सुंदर रूपात दिसेल: आणि मग SQL आणि NoSQL मधला आतला फरक कायमचा समजेल.

दोन प्रश्न प्रत्येक database ला: disk वर माहिती **कशी लिहायची** की (अ) शोधताना जलद सापडेल आणि (ब) लिहितानाही वेग राहील. या दोन इच्छांमध्ये सतत ताण, आणि जगात दोन मोठ्या युक्त्या आहेत: दोन वेगळे “सौदे.”

युक्ती 1: B-tree (SQL चं हृदय: वाचनासाठी बांधलेलं). सुव्यवस्थित ग्रंथालयाची उपमा: प्रत्येक पुस्तक त्याच्या ठरलेल्या, क्रमवार जागी. नवं पुस्तक आलं की **नेमक्या जागी** ठेवायचं (मध्ये सारून जागा करून). फायदा: शोधताना अचूक जागी, फार जलद (काही उड्यांत). किंमत: प्रत्येक नवं पुस्तक ठेवायला जागा करावी लागते: **लेखन थोडं हळू आणि विखुरलेलं** (disk वर इकडे- तिकडे जाऊन जागा करणं). B-tree = **वाचन जलद, लेखन थोडं महाग**. म्हणून जिथे वाचन जास्त, नाती-शोध जास्त: SQL/PostgreSQL इथे B-tree वापरतं.

युक्ती 2: LSM-tree (अनेक NoSQL चं हृदय: लेखनासाठी बांधलेलं). वेगळी युक्ती: नवं पुस्तक जागा शोधत बसू नको; **येईल तसं एका ढिगात, क्रमाने रचत जा** (फक्त शेवटी भर घाल: वेगवान). शोधायचं तेव्हा? अनेक ढिगांत बघावं लागेल (हळू). आणि मागाहून, सवडीने, ढिगांचं **एकत्रीकरण** (compaction) करून नीट लावायचं. फायदा: **लेखन प्रचंड जलद** (फक्त शेवटी भर). किंमत: वाचन थोडं हळू (अनेक ढिग), आणि compaction चा मागचा खर्च. LSM = **लेखन जलद, वाचन थोडं महाग**. म्हणून जिथे प्रचंड-वेगवान लेखन (sensor-data, logs, messages): Cassandra वगैरे NoSQL इथे LSM वापरतात.

आणि आता तो सुंदर, एकच सौदा दोन रूपांत:

B-tree (SQL): वाचन जलद <-> लेखन महाग. वाचन-जास्त कामाला.
LSM (NoSQL): लेखन जलद <-> वाचन महाग. लेखन-जास्त कामाला.

म्हणजे SQL-vs-NoSQL चा फरक फक्त “शिस्त-vs-सैल” नाही (2.2); तो **खोलात, disk वरच्या लेखन-वाचन सौद्याचा** आहे. आणि हा तोच index-चा सौदा (1.3: वाचन जलद, लेखन विकून), आता database च्या पायातच कोरलेला. software मध्ये **एकच सौदा पुन्हा-पुन्हा, वेगवेगळ्या पातळ्यांवर** दिसतो: हे ओळखता येणं म्हणजे खोल नजर.

हे तुम्हाला रोज बांधावं लागणार नाही (तयार databases हे करतात). मग का शिकायचं? कारण **database निवडताना** हा सौदा माहीत असला की तुम्ही बरोबर निवडता: “माझं app लेखन- जड की वाचन-जड?” या एका प्रश्नाने अर्धी निवड होते. आणि जेव्हा database “अचानक हळू” होते तेव्हा (उदा LSM ची compaction चालू असताना), कारण कळतं: अंधारात चाचपडत नाही.

महाग चूक

“Database आतून कशी चालते ते कशाला शिकायचं, तयार तर आहे.” तयार असलं तरी **निवड आणि निदान** तुमचं. LSM-database लेखन-जड कामाला निवडायची तिथे B-tree निवडली (किंवा उलट) की app झगडतं, आणि “database हळू आहे” म्हणत महागडं upgrade करत बसता: खरी अडचण चुकीच्या हृदयाची निवड असते. एका प्रश्नाचं (लेखन-जड की वाचन-जड) उत्तर हे 10x कामगिरीचा फरक ठरवू शकतं: तो प्रश्न विचारायला हे हृदय समजावं लागतं.

नकाशावर

- 2.1 data modeling 2.2 SQL vs NoSQL
- 2.3 DB चं हृदय: B-tree (वाचन जलद, SQL) vs LSM (लेखन जलद, NoSQL); तोच वाचन-वि.-लेखन सौदा, पायात कोरलेला

सूत्रावर: database च्या तळाशीही तोच एक सौदा: वाचन की लेखन. software मध्ये सौदे fractal आहेत: प्रत्येक पातळीवर तेच आकार, लहान-मोठे. एक सौदा नीट कळला की सगळे कळतात.

स्वतः बघा (5 मिनिटं)

दोन प्रकारे कपाट लावायचा विचार करा: (1) प्रत्येक वस्तू अकारविल्हे नेमक्या जागी (शोधायला सोपं, ठेवायला वेळ: B-tree); (2) येईल ती वस्तू पुढच्या रिकाम्या जागी टाका, महिन्याने एकदा सगळं आवरा (ठेवायला झटपट, शोधायला त्रास: LSM). तुम्ही घरात कोणती पद्धत वापरता, आणि का? तुमचं उत्तर तुमच्या “वाचन-जड की लेखन-जड” स्वभावावर अवलंबून: databases सुद्धा.

विचार करा

आरतीच्या app मध्ये एक भयंकर bug आला: दोन गिन्हाईकांनी **एकाच** क्षणी शेवटची एकच साखर-पोत ऑर्डर केली, आणि **दोघांनाही** “ऑर्डर घेतली” मिळालं: पण पोत एकच! Database कडक (SQL, ACID) असूनही असं कसं? आरती गोंधळली: “एकांत (Isolation, ACID मधलं I) म्हणजे तर दोन

व्यवहार मिसळत नाहीत ना? मग हे कसं घडलं?”

इथे ACID च्या सर्वात सूक्ष्म, सर्वात गैरसमजल्या भागाला हात लागला: race condition आणि transactions ची खरी खोली. “एकांत” म्हणजे नक्की किती एकांत, आणि तो कसा राखायचा: पुढचा chapter.

Chapter 2.4 [SPINE]: Transactions आणि Race Condition

आरतीचा “एकच पोतं, दोघांना ऑर्डर” हा bug software मधल्या सर्वात कपटी वर्गाचा आहे: **race condition** (शर्यत-दोष). आणि तो टाळणारं हत्यार म्हणजे transactions चं खरं आकलन.

Race condition म्हणजे काय. दोन माणसं एकाच क्षणी एकाच दाराने आत शिरायला बघतात: कोण आधी? “शर्यत” कोण जिंकतो यावर निकाल बदलतो: आणि तो अनिश्चित. आरतीच्या bug चं पडद्यामागचं दृश्य: दोन्ही ऑर्डर एकाच क्षणी database ला विचारतात “साखर आहे का?” -> दोघांनाही उत्तर “हो, 1 पोतं” (कारण अजून कुणीच घेतलं नाही) -> दोघेही “घेतो” म्हणतात -> दोघांनाही मिळालं. मधल्या **सूक्ष्म क्षणी** दोघांनी एकच “1 पोतं” बघितलं: हीच शर्यत. हे bugs भयंकर कारण ते **कधीतरीच** घडतात (नेमक्या एकाच क्षणी दोन विनंत्या आल्या तरच), म्हणून testing मध्ये दिसत नाहीत, आणि गर्दीच्या वेळीच फुटतात.

उपाय 1: Transaction (व्यवहार एकत्र बांधणं). ACID चं ‘A’ (Atomicity) आणि ‘I’ (Isolation) इथे कामाला. “साखर आहे का बघ + असेल तर घे + शिल्लक कमी कर”: या **तीन पायऱ्या एक अभंग गठ्ठा** म्हणून बांधायच्या (transaction). मग database हमी देते: हा गठ्ठा चालू असताना दुसरा कुणी मध्ये **घुसणार नाही** (Isolation), आणि गठ्ठा पूर्ण किंवा मुळीच नाही (Atomicity). बँकेची उपमा: “खात्यातून काढ + दुसऱ्यात टाक” हे दोन्ही किंवा काहीच नाही; मध्येच वीज गेली तरी अर्धवट नाही.

उपाय 2: कुलूप (locking). पण “मध्ये घुसणार नाही” कसं? Database एक **कुलूप** लावते: एक ऑर्डर साखरेच्या नोंदीवर काम करत असताना ती नोंद “कुलूपबंद”; दुसऱ्या ऑर्डरला **थांबावं** लागतं (कुलूप सुटपर्यंत). आता शर्यत संपली: एकच वेळी एकच. पहिल्याला पोतं मिळालं, दुसऱ्याला (थांबल्यावर) “संपलं” दिसलं: बरोबर.

पण इथे सूत्र, नेहमीप्रमाणे: कुलूप = शुद्धता विकत, वेग विकून. कुलूप लावलं की दुसऱ्याला **थांबावं** लागतं: म्हणजे हळू. आणि जास्त कुलूप = जास्त थांबे = कमी वेग. आणि एक भयंकर धोका: **deadlock** (परस्पर-कोंडी): ऑर्डर-अ ने साखरेचं कुलूप घेतलं आणि तेलाची वाट बघतोय; ऑर्डर-ब ने तेलाचं घेतलं आणि साखरेची वाट बघतोय: दोघेही एकमेकांची वाट बघत **कायमचे अडकले** (अरुंद पुलावर समोरासमोर दोन ट्रक: Book 6 त पुन्हा भेटेल). म्हणून database ना deadlock ओळखून एकाला “मागे फिर” सांगावं लागतं. निष्कर्ष: शुद्धतेसाठी कुलूप हवीत, पण जपून: जास्त कुलूप = हळू + deadlock चा धोका.

व्यावहारिक धडा: “एकाच वेळी अनेक लोक एकाच गोष्टीला हात लावतात” (शिल्लक, seat-booking, शेवटची वस्तू) तिथे race condition दबा धरून बसलेली असते. Top engineer तिथे **आधीच** transaction + योग्य कुलूप बांधतो; नवखा “असं दोघे एकाच क्षणी कसे येतील” म्हणत सोडतो, आणि

IPL-तिकीट/flash-sale च्या गर्दीत app कोसळतं. ”एकाच वेळी दोघे” हे नेहमी गृहीत धरा: Book 4 चं “अपयश गृहीत धरा” चं जुळं.

महाग चूक

”माझ्या testing मध्ये सगळं चाललं, bug नाही.” Race conditions हे testing ला सर्वात जास्त हुलकावणी देणारे bugs: कारण ते फक्त **नेमक्या एकाच क्षणी दोन विनंत्या** आल्या तरच दिसतात, आणि testing मध्ये तुम्ही एका वेळी एकच विनंती पाठवता. म्हणून “चाललं” म्हणजे “race नाही” नव्हे. जिथे सामायिक गोष्ट अनेकजण बदलतात तिथे transaction+lock **आधीच** बांधा, bug दिसायची वाट बघू नका: कारण तो गर्दीच्या, महत्त्वाच्या क्षणीच दिसेल: सर्वात महाग क्षणी.

नकाशावर

2.1-2.3 data modeling | SQL/NoSQL | B-tree/LSM
 2.4 transactions+race: “एकाच वेळी दोघे” गृहीत धरा; transaction (अभंग गट्टा) + lock (शुद्धता विकत, वेग+deadlock-धोका विकून)

सूत्रावर: शुद्धता (race टाळणं) फुकट नाही: कुलपं वेग खातात, deadlock चा धोका आणतात. “किती कडक एकांत हवा” हा सौदा: पैशाला कडक, like-count ला सैल.

स्वतः बघा (5 मिनिटं)

एका दाराची कल्पना करा जिथे एका वेळी एकच जण जाऊ शकतो. “कुलूप नसेल” तर दोघे धक्काबुक्की (race); “कुलूप असेल” तर एक थांबतो (सुरक्षित पण हळू). आता विचार करा: बँकेचं ATM एका वेळी एकच व्यवहार का करतं? कारण तुमच्या खात्यावर transaction+lock. तुम्ही रोज race-बचावाचा फायदा घेता, नकळत.

विचार करा

आरतीचं app आता अनेक सेवांचं बनलय: ऑर्डर-सेवा, payment- सेवा, delivery-सेवा. आणि प्रत्येक सेवेला दुसरीशी बोलावं लागतं. पण एका सेवेच्या टीमने आपल्या भागात बदल केला (एका माहितीचं नाव बदललं), आणि अचानक **दुसऱ्या** सेवेचं मोडलं: त्यांना कल्पनाही नव्हती. आरती वैतागली: “प्रत्येक सेवा दुसऱ्याशी बोलते, पण एकाने बदल केला की दुसरं फुटतं. या सेवांमध्ये काहीतरी **ठरलेला करार** नको का, जो एकतर्फी मोडता येणार नाही?”

हवाच: आणि त्या कराराचं नाव API. सेवांमधली सीमा, त्यांचा करार, आणि तो कसा टिकवायचा-
बदलायचा: पुढचा chapter, जो microservices च्या वादाचं दार उघडतो.

Chapter 2.5 [SPINE]: API: सेवांमधला करार

आरतीच्या “ठरलेला करार” या मागणीचं नाव **API** (Application Programming Interface). Book 2/4 मध्ये API भेटलं (यंत्राला हात); इथे ते सर्वात मूलभूत रूपात: **दोन software मधली सीमा आणि करार**.

API म्हणजे काय. hotel च्या menu-card ची उपमा: तुम्ही स्वयंपाकघरात घुसून “असं बनव” सांगत नाही; तुम्ही **menu** मधून मागता (ठरलेले पदार्थ, ठरलेली रीत). Menu हा तुमच्यातला आणि स्वयंपाकघरातला **करार**: तुम्हाला आत काय-कसं चालतं माहीत असायची गरज नाही (आचारी बदलला तरी menu तेच), आणि आचार्याला तुमची घाई सांभाळायची गरज नाही. API तेच: payment-सेवा म्हणते “मला असं विचारा (ऑर्डर-क्रमांक, रक्कम), मी असं उत्तर देईन (यशस्वी/अयशस्वी)”: हा करार. ऑर्डर-सेवेला payment च्या **आत** काय चालतं माहीत असायची गरज नाही.

API चा खरा फायदा: सीमा आणि स्वातंत्र्य. menu असेल तर आचारी आतली रेसिपी, भांडी, अगदी पूर्ण स्वयंपाकघर बदलू शकतो: **menu तेच राहिलं तर गिऱ्हाईकाला फरक पडत नाही**. API तेच देतं: करार (API) तोच ठेवला तर आतलं code, database, अगदी पूर्ण रचना बदलता येते, आणि वापरणाऱ्याला धक्का बसत नाही. आरतीच्या bug चं मूळ हेच होतं: एका टीमने **करारच** बदलला (menu मधला पदार्थ गायब केला), म्हणून दुसरं फुटलं. उपाय: **API हा पवित्र करार माना: तो एकतर्फी, अचानक बदलू नका**.

म्हणून API-रचनेचे नियम (करार लिहिण्याची कला): (1) **स्पष्ट आणि स्थिर**: काय मागायचं, काय मिळेल: नेमकं, बदलत नाही. (2) **जुनं तोडू नका (backward compatibility)**: नवं जोडायचं तर **जोडा**, जुनं काढू नका (जुने गिऱ्हाईक अजून जुनं वापरतात); menu त नवा पदार्थ घाला, जुना ठेवा. (3) **आवृत्त्या (versioning)**: मोठा बदल हवाच तर नवी आवृत्ती (v2), जुनी (v1) काही काळ चालू ठेवा, लोकांना हलायला वेळ द्या. (4) **चुका स्पष्ट सांगा**: “error” नाही, “ऑर्डर-क्रमांक चुकीचा” (Book 4, 4.2 चा शिकवणारा- error, इथे सेवांमध्ये). हे नियम पाळले की सेवा एकमेकांपासून **स्वतंत्र** होतात: एक बदलली तरी दुसरी शाबूत.

सूत्र इथे: API/करार = स्वातंत्र्य + स्पष्टता विकत, “पूर्ण लवचिकता” विकून. करार असेल तर आत काहीही बदला (स्वातंत्र्य), पण करार **पाळावाच** लागतो (बंधन). करार नसेल तर सगळं कुणालाही बदलता येतं (लवचिक), पण मग सगळे एकमेकांत गुंतून एकही सुरक्षित बदल अशक्य (आरतीचा bug). चांगली सीमा = चांगले शेजारी.

महाग चूक

”आतली गोष्ट आहे, दोन्ही सेवा आमच्याच, करार-बिरार कशाला: थेट एकमेकांची database वापरू.” हा सर्वात सामान्य, सर्वात घातक shortcut: एका सेवेने दुसरीच्या database ला **थेट** हात घातला की करार गेला, सीमा गेली: आता एकीचा कुठलाही आतला बदल दुसरीला अचानक फोडतो (आरतीचा bug कायमचा). नियम: **सेवांनी एकमेकांशी फक्त API ने बोलावं, थेट database ने कधीच नाही**. ही एक शिस्त म्हणजे microservices चालण्या-न-चालण्याची सीमारेषा (पुढचा chapter). Shortcut आज वेळ वाचवतो, उद्या महिने खातो.

नकाशावर

2.1-2.4 data modeling | SQL/NoSQL | B-tree/LSM | transactions
 2.5 API = सेवांमधला करार (menu); आत बदला, करार पाळा;
 जुनं तोडू नका, versioning; थेट database ला हात कधीच नाही

सूत्रावर: API हा “स्वातंत्र्य विरुद्ध बंधन” चा सौदा: करार पाळण्याचं बंधन घेतलं की आत बदलण्याचं स्वातंत्र्य मिळतं. चांगले करार चांगल्या रचनेचा पाया.

स्वतः बघा (5 मिनिटं)

कुठलंही app वापरताना “login with Google” बटण बघितलं असेल: ते app Google च्या **API** शी बोलतं (करार: “हा user खरा आहे का?”). App ला Google च्या आत काय चालतं माहीत नाही; Google ला app चं काही देणघेणं नाही: फक्त करार. एक API, दोन अनोळखी company, निर्दोष सहकार्य. करार-रचनेची ताकद तुम्ही रोज वापरता.

विचार करा

आता आरतीचा सर्वात मोठा प्रश्न: “करार (API) असतील तर मी app चे तुकडे करू शकते: ऑर्डर-सेवा, payment-सेवा, प्रत्येक वेगळी, वेगळी टीम. पण खरंच तुकडे करावेत का? एकाच मोठ्या code मध्ये सगळं (सोपं पण गर्दी) की अनेक वेगळ्या सेवा (स्वतंत्र पण गुंतागुंत)? सगळे म्हणतात microservices भारी: खरं का?”

हा Part 2 चा सर्वात मोठा, सर्वात गैरसमजलेला वाद: आणि इथे एका मोठ्या company ची **उलटी** गोष्ट आहे: Amazon ने microservices वरून परत monolith कडे जाऊन 90% खर्च वाचवला. Fashion विरुद्ध सौदा, सर्वात टोकाला. पुढचा chapter.

Chapter 2.6 [SPINE]: Monolith vs Microservices [दोन्ही बाजू + Amazon ची उलटी गोष्ट]

Part 2 चा सर्वात मोठा, सर्वात fashion-ग्रस्त वाद. आणि तो सोडवायला एक खरी, धक्कादायक गोष्ट: जिथे जगातल्या सर्वात मोठ्या company ने प्रवाहाच्या **उलट** जाऊन जिंकलं.

दोन रचना. (1) **Monolith (एकसंध):** पूर्ण app एकाच मोठ्या code मध्ये, एकत्र चालणारं. एका मोठ्या घराची उपमा: सगळ्या खोल्या एका छताखाली; आतल्या आत फिरणं सोपं-जलद, पण घर मोठं झालं की गर्दी. (2) **Microservices (अनेक सुटे):** app चे अनेक छोटे, स्वतंत्र सेवा-तुकडे, प्रत्येक वेगळा चालणारा, API ने बोलणारे (2.5). वेगवेगळ्या घरांची वसाहत: प्रत्येक स्वतंत्र, पण एका घरातून दुसऱ्यात जायला रस्ता-वाहन (network) लागतं.

Microservices काय देतं (आणि का सगळे भुलले): मोठ्या टीमला **स्वातंत्र्य**: ऑर्डर-टीम आपला तुकडा बदलते, payment-टीम आपला: एकमेकांच्या पायात न येता (2.4 चा आरतीचा वैताग सुटतो). प्रत्येक तुकडा वेगळा scale करता येतो (फक्त payment-सेवा गर्दीत वाढवा). एक तुकडा कोसळला तरी बाकी चालू (Book 4, 4.3 subagents चा दूरचा नातलग). 2015 च्या सुमारास हे इतकं गाजलं की “मोठं app = microservices” हे fashion झालं.

पण किंमत प्रचंड (जी fashion लपवते): एकाच घरात जे फुकट होतं (खोलीतून खोलीत चालणं) ते वसाहतीत **महाग** होतं: प्रत्येक सेवा-सेवा संवाद आता network वरून (हळू, latency: 1.9), फसू शकणारा (timeout/retry/circuit breaker: 1.11 आता **सगळीकडे** लागतं!). एका सोप्या कामाला दहा सेवांची साखळी: debugging भयंकर (कुठे बिघडलं?: observability अत्यावश्यक). आणि आरतीच्या race-सारखे प्रश्न आता सेवांमध्ये पसरतात (distributed transactions: भयंकर अवघड).

Microservices = टीम-स्वातंत्र्य विकत, प्रचंड गुंतागुंत विकून.

आणि आता ती उलटी गोष्ट (Amazon Prime Video, 2023): Amazon च्या Prime Video मधल्या एका टीमने त्यांची video- तपासणी सेवा microservices रचनेत बांधली होती (तुकडे, serverless). पण खर्च आणि गुंतागुंत वाढतच गेली. मग त्यांनी **उलट** केलं: तुकडे परत **एकाच process** मध्ये आणले (monolith कडे): आणि खर्च **90% घटला**, कामगिरी सुधारली. (नीट वाचा, कारण ही गोष्ट फार वेळा चुकीची सांगतात: **Amazon ने microservices सोडले नाहीत; एका टीमने एका सेवेला** तुकड्यांतून परत एकत्र आणलं, कारण त्या कामाला तुकड्यांची गरजच नव्हती, फक्त खर्च होता.) धडा सोन्याचा: **रचना fashion ने नाही, गरजेने.** त्या कामाला तुकड्यांचं स्वातंत्र्य नको होतं, म्हणून त्यांनी सौदा उलटवला: आणि प्रवाहाविरुद्ध जायची हिंमत हीच खरी अभियांत्रिकी.

निवडीची कसोटी (छापा):

Monolith निवडा:	छोटी-मध्यम टीम; सुरुवात; काम एकसंध; वेग आणि साधेपणा हवा. (बहुतेकांसाठी: हेच.)
Microservices निवडा:	मोठ्या अनेक टीमस एकमेकांच्या पायात येतात; भाग वेगवेगळे scale करावे लागतात; आणि गुंतागुंत पेलायची यंत्रणा (observability, अपयश-बचाव) आधीच आहे.
सोनेरी नियम:	"सुरुवात monolith ने; तुकडे तेव्हाच जेव्हा दुखणं (टीम-गर्दी) खरंच जाणवेल." उलट (आधीच microservices) = premature, महाग.

महाग चूक

"आम्ही आधुनिक आहोत, दिवस-1 पासून microservices." Startups ची सर्वात महाग fashion-चूक: 3 engineers, 100 users, आणि 15 microservices: प्रत्येक साध्या बदलाला दहा सेवांचा ताळमेळ, debugging नरक, आणि तुकड्यांचा फायदा (टीम-स्वातंत्र्य) शून्य (कारण टीमच लहान). Amazon-गोष्ट उलट शिकवते: गरज नसेल तर तुकडे **खर्च आहेत, फायदा नाही**. नियम: **monolith ने सुरू करा, दुखलं तर तोडा**. "तोडता येतं" (नंतर); "जोडणं" (परत एकत्र आणणं) महाग: म्हणून आधी एकत्र, नाईलाजाने वेगळं.

नकाशावर

2.1-2.5 data | SQL/NoSQL | B-tree/LSM | transactions | API
2.6 monolith vs microservices: micro = टीम-स्वातंत्र्य विकत,
गुंतागुंत विकून; Amazon Prime Video उलट गेलं (-90% खर्च);
सुरुवात monolith, तोडा गरजेला (fashion नको)

सूत्रावर: हा वाद म्हणजे सूत्राची सर्वात मोठी परीक्षा: सर्वात मोठा मोह (fashion), सर्वात मोठी लपलेली किंमत (गुंतागुंत). Amazon सारखं प्रवाहाविरुद्ध सौदा बघता येणं = सर्वोच्च नजर.

स्वतः बघा (5 मिनिटं)

तुमच्या घराचा विचार: सगळं एका घरात (monolith) सोयीचं की प्रत्येक कामाला वेगळी इमारत (microservices)? लहान कुटुंबाला एक घर पुरतं आणि सोपं; मोठ्या company ला वेगळे विभाग-इमारती लागतात, पण मग त्यांच्यात समन्वय-यंत्रणा (email, meetings = network+API). आकाराने

निर्णय बदलतो: तुमच्या app चा “आकार” ओळखा.

विचार करा

आरतीने monolith ठेवलं (शहाणपणाने). पण app मध्ये एक गरज वाढली: गिन्हाईकांना “साखर” search करता यायला हवं, आणि “साकर”, “sugar”, “साखरं” असं काहीही टाईप केलं तरी सापडायला हवं. तिचा नेहमीचा database (index सह) फक्त **नेमकं** जुळणारं शोधतो; “जवळपास” जुळणारं, स्पेलिंग-चुका सांभाळणारं शोधता येईना. “search तर index नेच होतं ना? मग हे वेगळं का?”

साधा शोध (index) आणि खरा “search” (जो अर्थ, स्पेलिंग, क्रम सांभाळतो) या वेगळ्या गोष्टी आहेत, आणि त्यामागे एक सुंदर रचना आहे: inverted index. पुढचा chapter: search आतून.

Chapter 2.7 [SPINE]: Search आतून: Inverted Index

आरतीचा प्रश्न: साधा शोध (index) आणि खरा search वेगळे कसे? उत्तर एका उलट्या युक्तीत आहे: आणि ती समजली की Google पासून app च्या search-बार पर्यंत सगळ्यामागचं तत्त्व कळतं.

साधा index वि. खरा search. database चा index (1.3): “रमेशचा फोन-नंबर -> ती नोंद.” हे **नेमकं जुळणं** शोधतं: अख्खा नंबर बरोबर हवा. पण search वेगळं मागतं: “साखर” टाइप केलं तर “साखरेचं पोतं”, “गूळ-साखर मिक्स”, “sugar free” : सगळं यायला हवं; स्पेलिंग-चूक (“साकर”) सांभाळायला हवी; आणि सर्वात जुळणारं **आधी** (क्रम). हे नेमक्या-जुळणीच्या index ला जमत नाही. यासाठी वेगळी रचना: **inverted index** (उलटा अनुक्रमणिका).

Inverted index म्हणजे काय (आणि “उलटा” का). सामान्य यादी: प्रत्येक वस्तू -> तिच्यातले शब्द (“पोतं-1: साखर, पांढरी, 1 किलो”). Search ला हे उलटं हवं: प्रत्येक **शब्द** -> तो कुठल्या वस्तूत आहे (“साखर -> पोतं-1, पोतं-5, ऑफर-3”). पुस्तकाच्या मागच्या index सारखं (1.3 ची उपमा, आता खरी): “साखर” शब्द शोधला की तो असलेल्या **सगळ्या** जागा एका झटक्यात. म्हणून “उलटा”: वस्तू-कडून-शब्द नाही, **शब्द-कडून-वस्तू**. आरती “साखर” टाइप करते -> inverted index मधून साखर असलेल्या सगळ्या वस्तू क्षणात.

खरा search जे जास्त करतो (तीन जादू): (1) **शब्द फोडणं आणि सुलभ करणं:** “साखरेचं”, “साखर”, “साखरं” : सगळं मुळाशी “साखर” वर आणणं (म्हणून वेगवेगळी रूपं जुळतात). (2) **स्पेलिंग-सहनशीलता:** “साकर” आणि “साखर” जवळचे, म्हणून जुळवणं (fuzzy). (3) **क्रम (ranking):** कुठलं आधी दाखवायचं: जास्त जुळणारं, जास्त लोकप्रिय, जवळचं दुकान: वर. Google चं संपूर्ण साम्राज्य या तिसऱ्या जादूवर (कुठलं पान आधी). यासाठी वेगळी search-यंत्रं असतात (Elasticsearch वगैरे): database नाही, search-साठीच बांधलेली.

सूत्र इथे: खरा search = लवचिक-हुशार शोध विकत, **जागा + गुंतागुंत + थोडा शिळेपणा** विकून. Inverted index वेगळी बांधावी- सांभाळावी लागते (जागा, यंत्रणा); आणि ती बहुधा मुख्य database पासून थोडी मागे असते (नवी वस्तू search मध्ये यायला क्षणभर: तोच replication-lag चा 1.5 चा आत्मा). म्हणून: **नेमकं जुळणं (फोन, क्रमांक) -> साधा index; लवचिक शोध (नाव, मजकूर) -> inverted index/search-यंत्र.** दोन वेगळी हत्यारं, दोन वेगळ्या गरजांना: पुन्हा तेच “माहितीच्या स्वभावाने निर्णय.”

महाग चूक

”Search हवा? मुख्य database वरच `LIKE %साखर%` ने शोधू” हा नवख्याचा सर्वात सामान्य फसवा shortcut: छोट्या data ला चालतो, पण तो **पूर्ण-चाळणी** (full scan: 1.3) असतो: लाखो नोंदीत प्रत्येक search म्हणजे अख्खी database चाळणं: app गुडघे टेकतं, आणि गर्दीत कोसळतं. खरा search ला inverted index/search-यंत्र लागतं: वेगळं, पण त्यासाठीच बांधलेलं. “database वर थेट LIKE search” हे flash-sale/गर्दीत मरणारं app चं सर्वात सामान्य मूळ आहे.

नकाशावर

2.1-2.6 data...monolith/microservices
 2.7 search: inverted index (शब्द -> वस्तू, उलटा); फोडणं + स्पेलिंग-सहन + क्रम(ranking); नेमकं->index, लवचिक->search-यंत्र

सूत्रावर: search ची हुशारी फुकट नाही: वेगळी यंत्रणा, जागा, थोडा शिळेपणा. “कुठला शोध हवा” (नेमका की लवचिक) ओळखून योग्य हत्यार निवडणं = नजर.

स्वतः बघा (5 मिनिटं)

Google वर मुद्दाम स्पेलिंग चुकवून शोधा (“जगातली सर्वात उंच इमारत”): Google “इमारत” समजून बरोबर देतं (fuzzy + ranking). आता फोन-contacts मध्ये अर्थ नाव टाईप करा: तेही जुळतं (inverted-सदृश). दोन्हीकडे “उलटा” शोध चालू आहे: शब्दाकडून-वस्तूकडे. एकदा हे दिसलं की search ची जादू यंत्र वाटते, चमत्कार नाही.

विचार करा

आरतीचं app आता आणखी एक गोष्ट मागतंय: delivery-boy कुठे आहे हे गिन्हाईकाला **live**, नकाशावर हलताना दिसायला हवं; आणि दुकान-गिन्हाईक यांच्यात **तात्काळ** chat. पण तिचं नेहमीचं request-response (गिन्हाईक विचारतो, server उत्तर देतो) यासाठी विचित्र वाटतं: गिन्हाईकाने दर सेकंदाला “boy कुठे?” विचारत राहायचं का? “सतत, आपोआप, दोन्ही बाजूंनी वाहणारी माहिती: हे request-response ने कसं जमणारं?”

जमणार नाही: request-response हे “विचारा-मग-उत्तर” चं तत्त्व; live/chat ला “सतत उघडा दुवा” लागतो. WhatsApp हे कोट्यवधी लोकांसाठी कसं करतं, तेच तत्त्व: पुढचा chapter: real-time.

Chapter 2.8 [SPINE]: Real-Time: WhatsApp चं तत्त्व

आरतीच्या live-tracking आणि chat च्या गरजेतून real-time communication ची संकल्पना. आणि इथे एक थक्क करणारी खरी गोष्ट आहे जी “मोठं म्हणजे प्रचंड टीम” हा गैरसमज मोडते.

Request-response ची मर्यादा. आतापर्यंतचं सगळं “गिन्हाईक विचारतो, server उत्तर देतो” (Book 1). पण chat/live-tracking वेगळं: server ला **स्वतःहून**, गिन्हाईकाने न विचारता, माहिती पाठवायची आहे (“boy आता इथे पोहोचला”, “नवा message आला”). दर सेकंदाला गिन्हाईकाने “काही नवीन?” विचारत राहणं (polling) म्हणजे दर मिनिटाला पोस्टमनला “पत्र आलं का?” विचारायला जाणं: वाया, महाग, आणि तरी उशिरा.

उपाय: सतत उघडा दुवा (persistent connection). पोस्टमनकडे जाण्याऐवजी घरात एक थेट **फोन-लाइन** ठेवायची: काही आलं की पोस्टमन **स्वतः** त्या लाइनवरून सांगतो. Software मध्ये याला WebSocket (आणि तत्सम) म्हणतात: गिन्हाईक आणि server मध्ये एक दुवा **उघडाच** राहतो; दोन्ही बाजू केव्हाही, एकमेकांना, ढकलू शकतात (push). नवा message आला की server लगेच ढकलतो; boy हलला की location लगेच पोहोचते: विचारावं लागत नाही. Chat, live-tracking, notifications: सगळं याच तत्त्वावर.

पण सूत्र, नेहमीप्रमाणे: उघडे दुवे = तात्काळपणा विकत, संसाधनं विकून. प्रत्येक उघडा दुवा server ची थोडी memory-लक्ष धरून ठेवतो. एक-दोन ठीक; पण **कोट्यवधी** लोक एकाच वेळी? कोट्यवधी उघडे दुवे सांभाळणं म्हणजे प्रचंड आव्हान: आणि इथे ती थक्क करणारी गोष्ट येते.

WhatsApp चं तत्त्व (खरी, तपासलेली गोष्ट): 2015 त WhatsApp **~90 कोटी** (900 million) लोकांना real-time chat देत होतं: कोट्यवधी उघडे दुवे, अब्जावधी messages रोज. आणि त्यांची engineering-टीम होती फक्त **~50 माणसं**. पन्नास. हे कसं? त्यांनी एक तत्त्व कट्टरपणे पाळलं: **सोपेपणा आणि योग्य हत्यार**. त्यांनी एका खास भाषेत (Erlang: लाखो सतत-दुवे सांभाळण्यासाठीच बनलेली) बांधलं, कमीत कमी features ठेवली, आणि प्रत्येक server कडून जास्तीत जास्त (एका server वर ~20 लाख दुवे) काढलं. धडा सोन्याचा (आणि Book 4, 9.3 शी थेट पूल): **मोठेपणा म्हणजे प्रचंड टीम/गुंतागुंत नव्हे; योग्य हत्यार + कट्टर सोपेपणा = 50 माणसं 90 कोटी लोकांना सेवा देतात.** आरतीसाठी दिलासा: तिलाही प्रचंड टीम लागणार नाही; योग्य निवडी लागतील.

महाग चूक

”Real-time हवं? सगळं WebSocket ने, सतत-उघडं करू!” उलट चूक: **सगळंच** real-time करणं म्हणजे विनाकारण कोट्यवधी उघडे दुवे, प्रचंड खर्च, गुंतागुंत. बहुतेक माहितीला real-time लागतच नाही: ऑर्डर-यादी, भाव: हे मागितल्यावर आलं तरी चालतं (request-response, स्वस्त). Real-time फक्त जिथे **तात्काळपणा खरंच महत्वाचा**: chat, live-location, alerts. नियम: **डिफॉल्ट request-response (स्वस्त, सोपं); real-time फक्त जिथे सेकंद मोजतात**. Fashion ने सगळं “live” करणारे app-मालक server- bill बघून रडतात.

नकाशावर

2.1-2.7 data...search

2.8 real-time: उघडा दुवा (WebSocket), server स्वतः ढकलतो;
तात्काळपणा विकत, संसाधनं विकून; WhatsApp 50 माणसं/90
कोटी = योग्य हत्यार + सोपेपणा; real-time फक्त जिथे सेकंद मोजतात

सूत्रावर: तात्काळपणा हा सर्वात मोहक अनुभव, पण संसाधन- महाग सौदा. आणि WhatsApp दाखवतं की सौदा हुशारीने केला तर मोठेपणा थोड्या माणसांतही पेलतो: सोपेपणा हाच सर्वात मोठा scaling-मंत्र.

स्वतः बघा (5 मिनिटं)

WhatsApp मध्ये कुणीतरी टाइप करतंय तेव्हा “typing...” दिसतं: तुम्ही काहीच विचारलं नाही, तरी माहिती **तुमच्याकडे आली** (server ने ढकलली: real-time). आता gmail उघडा आणि जुनी mail बघा: ती तुम्ही **मागितल्यावर** आली (request-response). दोन वेगळी तत्त्वं, दोन वेगळ्या गरजा. कुठे “ढकललं”, कुठे “मागितलं”: ओळखायला शिका.

विचार करा

आरतीचं app आता प्रचंड आणि गुंतागुंतीचं. आणि एक अपरिहार्य गरज आली: तिला एका जुन्या माहिती-रचनेत मोठा बदल करायचाय (2.1 चा निर्णय आता बदलायचाय), पण त्या रचनेत **लाखो** जिवंत नोंदी आहेत आणि app **चालूच** आहे: एक क्षणही बंद करता येत नाही. “चालत्या, भरलेल्या database ची रचना बदलायची, आणि app बंदही करायचं नाही: हे कसं शक्य आहे? चालत्या गाडीचं चाक कसं बदलणार?”

हाच Part 2 चा शेवटचा, आणि सर्वात नाजूक विषय: migrations. चालत्या system मध्ये बदल करण्याची कला, जिथे एक चूक लाखो नोंदी नष्ट करू शकते (GitLab ची रात्र आठवा). पुढचा chapter.

Chapter 2.9 [SPINE]: चालत्या गाडीचं चाक: Migrations

आरतीच्या “चालत्या database ची रचना बदलणं” या पेचावर हा chapter. आणि इथे software मधली सर्वात नाजूक कला आहे: कारण एक चूक = लाखो नोंदी नष्ट (GitLab ची रात्र, 1.2).

समस्येचं खरं रूप. Code बदलणं सोपं (जुनं फेकलं, नवं टाकलं). पण **लाखो जिवंत नोंदी** असलेल्या database ची रचना बदलणं (नवा रकाना, बदललेलं नातं, तुकडे करणं): हे धावत्या रेल्वेचे रूळ बदलण्यासारखं: गाडी चालूच आहे, प्रवासी आतच आहेत, आणि थांबवता येत नाही. याला **migration** म्हणतात, आणि त्याचं सोनेरी तत्त्व एकच: **कधीच एका मोठ्या उडीने नाही; नेहमी छोट्या, उलटवता-येणाऱ्या पावलांनी.**

सुरक्षित migration ची पद्धत (छापून ठेवा): विस्तार-मग-आकुंचन. समजा आरतीला एका रकान्याचं नाव/रचना बदलायचीय. **चुकीचा मार्ग:** एका रात्री सगळं एकदम बदलणं (जुनं काढून नवं): चुकलं तर सर्वनाश, आणि जुनं-code अजून जुनं-रकाना शोधत असेल तर लगेच सगळं फुटतं. **बरोबर मार्ग, चार पावलं:**

1. विस्तार: नवा रकाना जोडा (जुना ठेवूनच). आता दोन्ही आहेत. काहीच मोडत नाही (जुनं code जुनं वापरतं).
2. दुहेरी-लेखन: नवं code दोन्हीकडे लिहितं (जुना + नवा), जुनी माहिती हळूहळू नव्यात कॉपी करते. app चालूच.
3. वळण: सगळं तयार झाल्यावर code ला नवा रकाना वापरायला सांगा. तपासा. चुकलं तर लगेच जुन्याकडे परत (सुरक्षित).
4. आकुंचन: सगळं नव्यावर स्थिरावल्यावर, खात्री पटल्यावर, जुना रकाना काढा. आता स्वच्छ.

प्रत्येक पाऊल **छोटं आणि उलटवता-येणारं**: कुठल्याही क्षणी चुकलं तर मागे फिरता येतं, app बंद पडत नाही, नोंदी सुरक्षित. हे मंद वाटतं (चार पावलं, दिवस-आठवडे), पण तेच वाचवतं: घाईची एक मोठी उडी = GitLab ची रात्र.

आणि दोन लोखंडी नियम (Book 4, 4.5 चे भाऊ): (1) **आधी backup**, आणि तो तपासलेला (न तपासलेला backup = नसलेला backup: GitLab). (2) **आधी थोड्यांवर, मग सगळ्यांवर** (एका छोट्या भागावर migration चालवून बघा, मग पूर्ण: CrowdStrike चा 85-लाख-यंत्रांचा धडा, 1.13). Migration म्हणजे शस्त्रक्रिया: घाई नाही, तयारी सगळी, आणि परत-फिरण्याचा रस्ता नेहमी उघडा.

सूत्र इथे: सुरक्षित migration = जोखीम-घट विकत, वेग विकून. एका रात्रीची उडी जलद पण जीवघेणी; चार-पावलांचा रस्ता मंद पण सुरक्षित. आणि इथे सौदा नेहमी सुरक्षेच्या बाजूने: कारण वेगाची किंमत “थोडा उशीर”, जोखमीची किंमत “लाखो नोंदी + धंदा.” असला सौदा एकतर्फीच.

महाग चूक

“छोटा बदल आहे, रात्री एकदम करून टाकू, कोण बघतंय.” Migrations मधली सर्वात महाग, सर्वात सामान्य चूक: “छोटा बदल” लाखो नोंदींवर चालतो, आणि एक चुकीची ओळ सगळं पुसते (GitLab चा engineer “छोटं” काम करत होता). आणि रात्री-एकट्याने-घाईत म्हणजे backup नाही, तपासणी नाही, परत-फिरण्याचा रस्ता नाही. नियम: ****कुठलाही database-बदल = शस्त्रक्रिया = विस्तार-आकुंचन**

- backup + थोड्यांवर-आधी. **** “छोटा बदल” असं database-जगात काही नसतं; लाखो नोंदींवर सगळंच मोठं.**

नकाशावर

2.1-2.8 data...real-time

2.9 migrations: चालत्या गाडीचं चाक; विस्तार->दुहेरी-लेखन-> वळण->आकुंचन (छोटी, उलटवता-येणारी पावलं); backup तपासलेला + थोड्यांवर आधी; जोखीम-घट विकत, वेग विकून

सूत्रावर: इथे सूत्र सुरक्षेकडे झुकतं: काही सौदे एकतर्फी असतात. लाखो नोंदींशी खेळताना “वेग विरुद्ध सुरक्षा” मध्ये सुरक्षा नेहमी जिंकते: कारण चुकीची किंमत भरून येत नाही.

स्वतः बघा (5 मिनिटं)

घरातलं जुनं कपाट नव्या जागी हलवायचा विचार करा: सगळं एकदम उचलून टाकणं (काहीतरी फुटेल) vs एक-एक कप्पा हलवत, जुनं रिकामं होईपर्यंत नवं भरत जाणं (सुरक्षित, मंद). Migration ची विस्तार-आकुंचन पद्धत नेमकी दुसरी: जुनं-नवं काही काळ दोन्ही, मग हळूच जुनं काढणं. रोजच्या आयुष्यातही सुरक्षित बदलाचं तत्त्व तेच.

विचार करा

Part 2 संपत आलाय. आरतीने आता खूप शिकलंय: data-रचना, database-निवड, transactions, API, monolith/microservices, search, real-time, migrations. पण एक कौशल्य उरलंय जे या सगळ्याला जोडतं: “जेव्हा मला कुणाच्या तरी system चा नकाशा (architecture diagram) दाखवला जातो, तो **वाचता** यायला हवा: यात कुठे cache आहे, कुठे धोका आहे, कुठला सौदा केलाय: हे एका नजरेत ओळखता यायला हवं. ते कसं शिकायचं?”

हाच Part 2 चा कळस: सगळे तुकडे एकत्र बघून “नकाशा वाचणं.” कारण top engineer ची खरी ओळख नवं बांधण्यात इतकीच दुसऱ्याची रचना **वाचून सौदे-धोके ओळखण्यात** आहे. विभागाचा शेवटचा chapter.

Chapter 2.10 [SPINE]: Architecture Diagram वाचण्याची कला

Part 2 चा कळस: दुसऱ्याची रचना **वाचणं**. कारण top engineer ची अर्धी किंमत नवं बांधण्यात, अर्धी दुसऱ्याचा नकाशा बघून **सौदे आणि धोके एका नजरेत ओळखण्यात** आहे. हे कौशल्य मुलाखतीत, code-review त, आणि रोजच्या निर्णयांत सतत लागतं.

नकाशा म्हणजे काय. architecture diagram: पेट्या (सेवा, database, cache) आणि बाण (कोण कुणाशी बोलतं). Part 1 चं “आरतीचं पूर्ण चित्र” (1.15) हाच एक नकाशा होता. आता तो **वाचायचा**: म्हणजे पेट्या-बाण बघून मागचे सौदे-धोके ताडायचे.

वाचनाची सहा प्रश्नांची नजर (छापून ठेवा):

1. एका विनंतीचा रस्ता कुठला? बोट ठेवून पूर्ण प्रवास काढा: कुठून येते, कुठे-कुठे जाते, उत्तर कसं परत. (1.1, 1.15)
2. एकच अपयश-बिंदू कुठे? एक पेटी पडली तर सगळं थांबतं अशी जागा आहे का? (एकच database, एकच सेवा सगळे वापरतात: तिथे धोका.) (1.1, 1.8)
3. डेटा कुठे, कुठली database? पैसे SQL त आहेत ना? सैल-प्रचंड माहिती NoSQL त? की सगळं एकाच कॉबलंय? (2.2)
4. Cache कुठे, “जुनं कधी”? cache दिसतंय का? मग शिळेपणाचा प्रश्न विचारला गेलाय का? (1.4)
5. सेवा-सेवा कशा बोलतात? API ने (करार, सुरक्षित) की एकमेकांच्या database ला थेट हात (धोका!)? (2.5)
6. गर्दी/अपयश कुठे फुटेल? load balancer आहे? queue आहे (धक्का शोषायला)? timeout/retry? गर्दीत कुठली पेटी आधी पडेल? (1.7, 1.8, 1.11)

ही सहा-प्रश्नांची नजर कुठल्याही नकाशाला लावा: दोन मिनिटांत तुम्ही रचनेची ताकद आणि तडे दोन्ही सांगू शकाल. हेच “वाचता येणं.”

आणि सर्वात प्रौढ प्रश्न: सौदे कुठे केलेत, आणि ते बरोबर आहेत का? प्रत्येक नकाशा म्हणजे सौद्यांचा गट्टा (पूर्ण Part 1-2). नकाशा वाचताना विचारा: “इथे cache आहे म्हणजे यांनी वेग निवडला, ताजेपणा सोडला: या माहितीला ते चालेल का? इथे microservices आहेत म्हणजे टीम-स्वातंत्र्य निवडलं, गुंतागुंत घेतली: यांची टीम एवढी मोठी आहे का?” **सौदा दिसणं आणि तो या परिस्थितीला योग्य आहे का हे ताडता येणं** : हीच पूर्ण Book 5 ची कमाई, एका कौशल्यात.

हे कुठे वापराल: (1) **मुलाखत:** “design a system like X”: आता तुम्ही पेढ्या-बाण काढून सौदे बोलू शकता (हेच system- design मुलाखतीचं हृदय). (2) **नवं काम:** दुसऱ्याने बांधलेलं समजून घ्यायला. (3) **AI-युगात (Book 4 शी पूल):** agents code लिहितील, पण **रचना बरोबर आहे का, सौदे योग्य आहेत का** हे ठरवणारी नजर तुमची: आणि तीच leverage (Book 4, 9.2: निर्णय-नेमकेपणा). Code टाईप करणं स्वस्त झालं; **नकाशा वाचणं- आखणं** महाग राहिलं.

महाग चूक

”नकाशा भारी-गुंतागुंतीचा दिसतोय, म्हणजे यांनी चांगलं काम केलंय.” उलट बहुधा खरं: **गुंतागुंत हे कौशल्याचं लक्षण नाही, अनेकदा तो न-विचारलेल्या सौद्यांचा ढीग असतो.** खरा जाणकार सर्वात **सोपी** रचना शोधतो जी काम करते (WhatsApp चं 50- माणसं तत्त्व, 2.8; Amazon चं परत-monolith, 2.6). नकाशा वाचताना “किती भारी” ने भारावू नका; “कमीत कमी तुकड्यांत काम होतंय का, प्रत्येक तुकडा **कशासाठी** आहे” विचारा. जिथे एखादी पेटी “का आहे” चं उत्तर नाही, तिथे बहुधा fashion आहे, गरज नाही.

नकाशावर

Part 2 पूर्ण:

data modeling | SQL/NoSQL | B-tree/LSM | transactions |
API | monolith/microservices | search | real-time | migrations
2.10 नकाशा वाचणं: सहा प्रश्न (रस्ता, अपयश-बिंदू, database, cache, सेवा-संवाद, गर्दी); सौदा दिसणं + योग्य का ताडणं

सूत्रावर: पूर्ण Part 2 एका कौशल्यात उतरतं: नकाशा वाचणं म्हणजे **सौदे वाचणं**. आणि सौदे वाचता येणं म्हणजे रचना बांधता येणं: कारण दोन्ही एकच नजर.

स्वतः बघा (5 मिनिटं)

Book 5 च्या 1.15 मधलं “आरतीचं पूर्ण चित्र” परत उघडा आणि वरची सहा-प्रश्नांची नजर त्याला लावा: एका विनंतीचा रस्ता काढा, अपयश-बिंदू शोधा, cache कुठे बघा. दोन मिनिटांत तुम्ही तुमचाच नकाशा “वाचाल”: आणि हेच कौशल्य उद्या दुसऱ्याच्या (किंवा मुलाखतीतल्या) नकाशाला लागेल.

विचार करा

Part 2 संपला. आरतीकडे आता रचनेचे सगळे निर्णय आहेत, आणि ते वाचायची नजरही. पण एक गोष्ट उरलीये, आणि तीच कदाचित सर्वात महत्वाची: “हे सगळं मला **एकटीला** येतं. पण मोठं software 30-50 लोक **मिळून** बांधतात. ते एकमेकांच्या पायात न येता, चुका न करता, वेगात कसे बांधतात? WhatsApp चे 50 लोक, Zerodha चे 30: त्यांची **काम करण्याची पद्धत** काय आहे? रचना कळली; पण **elite टीम** कशी काम करते?”

हाच Part 3 चा विषय, आणि तो रचनेइतकाच महत्वाचा: git, testing, review, incident-संस्कृती: elite teams ची शिस्त, आणि मग WhatsApp/UPI/Zerodha/Netflix च्या खऱ्या इमारती उलगडून. Part 3: शिस्त आणि खऱ्या इमारती.

BOOK 5, PART 2 चा शेवट

जे तुमच्याकडे आता आहे

data modeling	एक सत्य एका जागी (normalize) vs वेगासाठी कॉपी (denormalize); नाती आधी; रचना आधी
SQL vs NoSQL	SQL=शिस्त+नाती, NoSQL=आकार+वेग; डिफॉल्ट SQL, NoSQL गरजेला (fashion नको)
B-tree vs LSM	वाचन-जलद (SQL) vs लेखन-जलद (NoSQL); तोच वाचन/लेखन सौदा, पायात
transactions/race	"एकाच वेळी दोघे" गृहीत धरा; transaction+lock (शुद्धता विकत, वेग+deadlock विकून)
API	सेवांमधला करार (menu); आत बदला, करार पाळा; थेट database ला हात कधीच नाही
monolith/micro	micro=टीम-स्वातंत्र्य विकत, गुंतागुंत विकून; Amazon उलट गेलं (-90%); सुरुवात monolith
search	inverted index (शब्द->वस्तू); नेमकं->index, लवचिक->search-यंत्र; database वर LIKE नको
real-time	उघडा दुवा; तात्काळपणा विकत, संसाधनं विकून; WhatsApp 50/90कोटी = योग्य हत्यार+सोपेपणा
migrations	विस्तार->दुहेरी-लेखन->वळण->आकुंचन; backup तपासलेला; थोड्यांवर आधी; जोखीम-घट विकत
नकाशा वाचणं	सहा प्रश्न; सौदा दिसणं + योग्य का ताडणं = Book 5 ची कमाई

स्वतःची परीक्षा (4 प्रश्न, बोलून; उत्तरं खाली)

1. नवं app: database डिफॉल्ट SQL की NoSQL, आणि का?

SQL: नाती+शुद्धता फुकट मिळतात, नंतर सोडता येतात; NoSQL ची खरी गरज (प्रचंड-सैल-वेगवान) असेल तरच तिकडे. Fashion नको. (2.2)

2. दोन सेवा आमच्याच आहेत; एकीने दुसरीच्या database ला थेट हात घातला तर?

करार (API) तुटला, सीमा गेली: आतला कुठलाही बदल दुसरीला फोडेल. सेवांनी फक्त API ने बोलावं. (2.5)

3. Amazon Prime Video ची गोष्ट काय शिकवते?

रचना fashion ने नाही, गरजेने: त्यांनी microservices वरून परत monolith कडे जाऊन 90% खर्च वाचवला. प्रवाहाविरुद्ध सौदा बघणं = नजर. (2.6)

4. चालत्या database ची रचना बदलायची: पद्धत?

विस्तार-आकुंचन: नवं जोडा (जुनं ठेवून) -> दुहेरी-लेखन -> वळण -> जुनं काढा; प्रत्येक पाऊल छोटं, उलटवता-येणारं; backup आधी. (2.9)

पुढच्या भागाची चाहूल

आरतीचा शेवटचा प्रश्न Part 3 उघडतो: **elite टीम कशी काम करते?** git (इतिहास + review), testing चे थर, CI/CD, debugging = निदान-कला, blameless incident-संस्कृती, आणि सुरक्षित deploy (canary). मग खऱ्या इमारती उलगडून: WhatsApp, UPI, Zerodha, Netflix: आणि AI-युगात हे सगळं का उरतं. Part 3: शिस्त आणि खऱ्या इमारती.

BOOK 5, PART 2 चा MINI-GLOSSARY

API	दोन software मधला करार (menu-तत्त्व) (2.5)
B-tree	वाचन-जलद database-रचना (SQL) (2.3)
deadlock	दोन व्यवहार एकमेकांची वाट बघत अडकणं (2.4)
denormalization	वेगासाठी माहिती मुद्दाम कॉपी करणं (2.1)
inverted index	शब्द -> वस्तू; खऱ्या search चा पाया (2.7)
LSM-tree	लेखन-जलद database-रचना (NoSQL) (2.3)
migration	चालत्या database ची रचना सुरक्षित बदलणं (2.9)
monolith	पूर्ण app एकाच code मध्ये (2.6)
microservices	app चे अनेक स्वतंत्र सेवा-तुकडे (2.6)
normalization	एक सत्य एका जागी; पुनरावृत्ती टाळणं (2.1)
NoSQL	सैल-लवचिक, आकार-वेगासाठी database (2.2)
race condition	एकाच वेळी दोघांनी हात घातल्याने अनिश्चित निकाल (2.4)
SQL/relational	शिस्त+नाती+ACID असलेली database (2.2)
transaction	अनेक पायऱ्या एक अभंग गट्टा (2.4)
versioning	API ची नवी आवृत्ती, जुनी चालू ठेवून (2.5)
WebSocket	सतत-उघडा दुवा; real-time साठी (2.8)